Building the DrugCLIP combine_set Candidate-Pocket Library

Architecture, data contracts, provenance, validation, and operating guide

BioSensIA

2026-07-28

Table of contents

1 Purpose

This document explains the second candidate-pocket library implemented in this repository. The first library reconstructs pocket geometry from licensed PDBbind protein and ligand structures. The second library starts from the bundles below DrugCLIP's vendored directory:

```
external/DrugCLIP/data/pdb/combine_set/
   <pdb-id>/
       data.pkl
       <pdb-id>_ligand.sdf
       <pdb-id>_ligand.mol2
       <pdb-id>_protein.pdb
       <pdb-id>_pocket.pdb
       <pdb-id>_pocket6A.pdb
```

The goal is not simply to copy pickle files into an LMDB. A usable candidate library must also provide stable identity, source checksums, ligand identity, chain and residue provenance, explicit mapping uncertainty, Parquet sidecars, LMDB checksums, and validation across all artifacts.

The implemented products are:

1. an explicit-schema sidecar collection containing scientific data and provenance;
2. a DrugCLIP-compatible candidate-pocket LMDB;
3. profile metadata containing contract and artifact checksums;
4. a manifest, stage checkpoints, logs, and build reports.

The operational commands are deliberately separate from the original PDBbind commands:

```
uv run biosensia-pocket-library inventory-combine-set \
  --config config/drugclip-combine-set-library.toml \
  --pdb-id 2h2h

uv run biosensia-pocket-library build-combine-set \
  --config config/drugclip-combine-set-library.toml \
  --pdb-id 2h2h
```

2 Why the design changed after inspecting the data

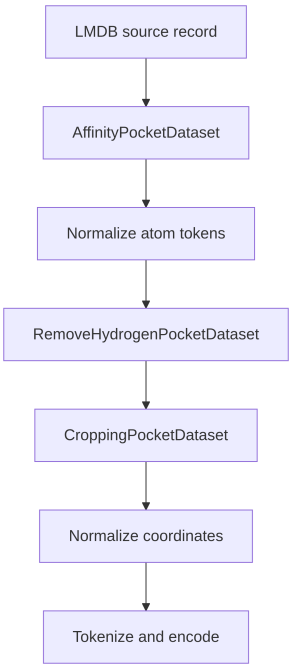
2.1 Initial assumption

The initial design assumed that `data.pkl` might already contain the final pocket representation expected by the model: hydrogen-free, bounded to the configured maximum number of atoms, and ready to serialize directly through the existing post-transform LMDB path.

That assumption was incorrect.

2.2 Observed DrugCLIP behavior

The pickles contain an **input-stage representation**. They commonly retain hydrogen and can contain substantially more than the default 256 pocket atoms. The vendored DrugCLIP target-fishing path performs the following operations after reading an LMDB record:



Therefore, removing hydrogen or cropping while building this second LMDB would apply part of the DrugCLIP transformation twice, or would replace DrugCLIP's established crop with a different representation. Either change would make the new candidate library scientifically different from the original combine-set input.

2.3 Revised rule

The revised implementation distinguishes two record representations:

Representation	Producer	Hydrogen	Atom limit at serialization	Later DrugCLIP transform
`post_transform`	PDBbind re-extraction builder	Excluded	Enforced	No additional atom removal should be needed

Representation	Producer	Hydrogen	Atom limit at serialization	Later DrugCLIP transform
`source_pickle`	combine-set builder	Preserved	Not enforced	DrugCLIP removes hydrogen and crops

This distinction is recorded in `lmdb_records.record_representation`. It is also used by the exporter and validator so that a raw source record is not rejected for correctly containing hydrogen or more than 256 atoms.

The invariant for the combine-set builder is:

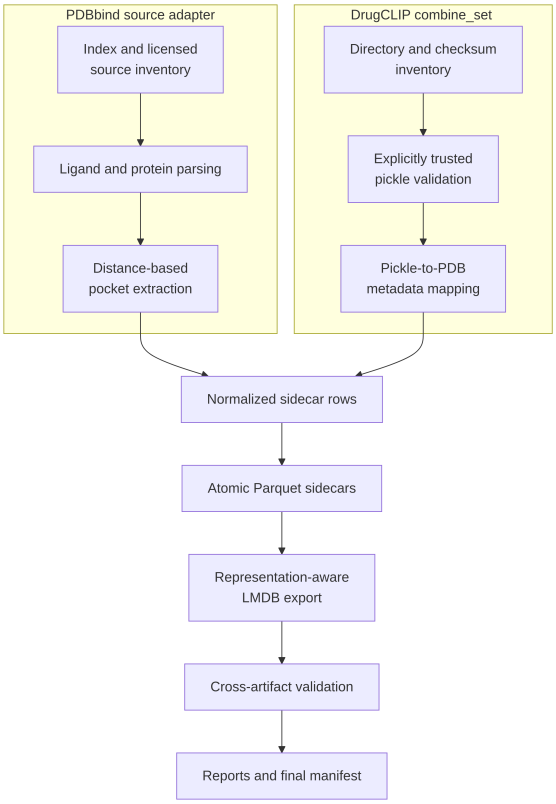
> The ordered `pocket_atoms` sequence and corresponding `pocket_coordinates` array written to LMDB are the validated source-pickle representation, converted only to contiguous little-endian float32 coordinates for a deterministic serialization contract.

3 Architecture

3.1 Source adapter instead of a second independent application

The implementation adds a source-specific ingestion path but reuses the artifact-producing core. This avoids two undesirable extremes:

- inserting combine-set conditions throughout the PDBbind geometry extractor;
- copying the entire pipeline and allowing validation or serialization behavior to diverge over time.



3.2 Module map

Module	Responsibility
`trusted_pickle.py`	Explicit trust gate, pickle loading, field/type/shape/finiteness validation, DrugCLIP token transformation
`combine_set_mapping.py`	Exact ordered and spatial atom mapping from pickle geometry to neighboring pocket PDB files
`combine_set_source.py`	Discovery, selection, source inventory, stable identities, and normalized row production
`combine_set_pipeline.py`	Run orchestration, stage checkpoints, optional RCSB enrichment, sidecars, LMDB, validation, and reports
`rcsb.py`	Content-addressed download plus checksum, compression, and mmCIF-identity validation; cache-only resolution performs no network I/O
`rcsb_enrichment.py`	Source-neutral application of verified mmCIF mappings, UniProt links, ligand candidates, and structural citations
`rcsb_workflow.py`	Deterministic request planning, online prefetch manifests, and immutable cache-only derived runs
`schemas.py`	Additive source-neutral sidecar schema v2
`sidecars.py`	Atomic writes plus compatibility reads for completed schema-v1 runs
`lmdb_export.py`	`source_pickle` and `post_transform` serialization contracts
`validation.py`	Representation-aware content hashes, joins, enums, and LMDB validation
`cli.py`	Dedicated inventory/build commands and the source-neutral post-build RCSB workflow

The original `pipeline.py`, ligand parser, DrugCLIP contract verifier, hashing, manifest, sidecar, reporting, and LMDB modules remain shared.

4 Trust boundary

4.1 Why pickle requires an explicit decision

Python pickle is not a passive data format. Deserializing a malicious payload can execute code. Checking a file's extension, placing it in a subprocess, or computing its SHA-256 digest does not make it safe.

The builder therefore refuses to deserialize `data.pkl` unless trust is explicitly authorized:

```
[combine_set]
trusted_pickles = true
```

The same authorization can be made for one CLI invocation:

```
uv run biosensia-pocket-library build-combine-set \
  --config config/drugclip-combine-set-library.toml \
  --trust-pickles \
  --pdb-id 2h2h
```

This option must be enabled only for the known vendored DrugCLIP snapshot. It must not be reused for downloaded or user-supplied pickle files without a separate trust decision.

4.2 Inventory is safe before deserialization

`inventory-combine-set` performs directory discovery, file classification, size checks, and SHA-256 hashing. It does not call `pickle.load`. This makes it possible to audit membership and provenance before authorizing executable serialization.

During a build, source inventory and hashing also happen before record processing. The resulting manifest records:

- the source kind;
- the selected record count;
- every selected file checksum;
- the aggregate source fingerprint;
- the fact that trusted pickle execution was authorized.

5 Discovery, selection, and source inventory

5.1 Discovery contract

Discovery searches only direct children of the configured combine-set root and retains a directory only when it contains a regular `data.pkl` file. Records are sorted by normalized directory identifier. Duplicate identifiers are an error because a four-character directory name must identify one occurrence in this source snapshot.

On the source snapshot used during implementation, discovery found 18,938 directories containing `data.pkl`. This is a snapshot observation, not a hard-coded expected count. The selected file hashes, not the count alone, define source identity.

5.2 Deterministic selection

Selection is applied after sorting:

```
# One named occurrence
uv run biosensia-pocket-library inventory-combine-set \
  --config config/drugclip-combine-set-library.toml \
  --pdb-id 2h2h

# First ten sorted occurrences
uv run biosensia-pocket-library inventory-combine-set \
  --config config/drugclip-combine-set-library.toml \
  --limit 10

# IDs from a text file, one per line
```

```
uv run biosensia-pocket-library build-combine-set \
  --config config/drugclip-combine-set-library.toml \
  --pdb-ids-file selected-pdb-ids.txt
```

Unlike the index-driven PDBbind source, this source has no release-year field that can support `--year-from` or `--year-to`. Those options are intentionally absent from the combine-set commands.

5.3 File roles

Each selected directory is inventoried completely. Expected filenames receive specific source kinds:

Filename	Source role	Primary use
`data.pkl`	`pickle`	Authoritative LMDB pocket geometry and source label
`*_ligand.sdf`	`ligand_sdf`	Primary ligand chemistry and geometry
`*_ligand.mol2`	`ligand_mol2`	Ligand fallback and cross-format comparison
`*_protein.pdb`	`protein_pdb`	Protein provenance and future structure diagnostics
`*_pocket.pdb`	`pocket_pdb`	Candidate source for atom-to-residue mapping
`*_pocket6A.pdb`	`pocket6a_pdb`	Alternative candidate source for atom-to-residue mapping
any other file	`extra`	Retained inventory provenance

A missing neighboring file creates a processing issue. A missing SDF and MOL2 together prevents ligand construction. A missing pocket PDB does not change the pickle geometry, but it can leave chain and residue mapping unresolved.

6 Trusted pickle validation

6.1 Required pocket fields

The loader requires a dictionary with these fields:

```
required = {
    "pocket",
    "pocket_atoms",
    "pocket_coordinates",
}
```

The `pocket` value must be nonempty and must agree case-insensitively with the directory identifier. `pocket_atoms` must be a nonempty sequence of nonempty tokens. `pocket_coordinates` must represent exactly one finite numeric `(n_atoms, 3)` coordinate array, and both fields must have equal lengths.

Ligand-side `atoms` and `coordinates` fields are also shape-checked when present. They are retained as source audit information, while chemical identity comes from the neighboring SDF/MOL2 parser.

6.2 Why coordinate layouts are normalized carefully

DrugCLIP data can represent ligand coordinates as more than one conformer. A plain `numpy.asarray(value).reshape(...)` would be dangerous because it could silently merge conformers or change atom grouping.

The loader accepts only explicit layouts:

- one `(atoms, 3)` numeric array;
- a numeric `(conformers, atoms, 3)` array;
- an iterable whose individual items are `(atoms, 3)` arrays.

Pocket geometry must resolve to exactly one coordinate set. Ligand conformers do not create multiple pocket instances.

6.3 DrugCLIP token normalization

The active DrugCLIP `AffinityPocketDataset` reduces each source atom token to a single dictionary token:

```python
def drugclip_pocket_token(atom: str) -> str:
    if atom[0].isdigit() and len(atom) > 1:
        return atom[1]
    return atom[0]
```

For example:

Source token	DrugCLIP token
`C`	`C`
`H`	`H`
`1C`	`C`

Validation checks the transformed tokens against the fingerprinted `dict_pkt.txt`. The raw source tokens remain in the LMDB because DrugCLIP owns the transformation. This checks compatibility without changing the source representation prematurely.

7 Ligand identity and geometry

The pickle does not provide a complete, validated ligand identity contract. The builder therefore reuses the established ligand parser:

1. try the configured primary format, normally SDF;
2. fall back to MOL2 if necessary;
3. require finite three-dimensional coordinates and resolved atomic numbers;
4. retain disconnected component membership;
5. derive SMILES, InChI, InChIKey, formula, charge, and molecular weight where RDKit can do so reliably;
6. compare independently usable SDF and MOL2 structures;
7. mint ligand content and derivation hashes.

The parsed ligand has two roles in this builder:

- it provides auditable chemical identity;
- its heavy-atom coordinates support diagnostic pocket-to-ligand distances.

It does **not** redefine membership of the pickle pocket.

8 Mapping pickle atoms to structure metadata

8.1 Geometry and metadata have different authority

The pickle is authoritative for model input geometry. The neighboring PDB is a metadata source for atom names, residues, chain identifiers, occupancy, and B-factors. Mapping can annotate a pickle atom, but it cannot replace its token or coordinates.

This separation prevents a plausible neighboring file from silently becoming the geometry source merely because it shares a PDB ID.

8.2 Candidate files

Both `*_pocket.pdb` and `*_pocket6A.pdb` are considered. Each is mapped independently. The preferred mapping is selected by:

1. greatest mapped-atom count;
2. smallest maximum coordinate error;
3. stable lexical source-role tie-break.

The selected role is written to the comparison sidecar. The v2 comparison schema also names both sides explicitly:

```
left_geometry_role  = drugclip_combine_set_pickle
right_geometry_role = pocket | pocket6a | neighbor_pocket_unavailable
```

8.3 Exact ordered mapping

The strongest mapping occurs when all of the following hold:

- the PDB and pickle have equal atom counts;
- transformed pickle elements equal PDB elements in the same order;
- every coordinate error is within `combine_set.compare_coordinate_tolerance_angstrom`.

Each atom then receives `source_mapping_status = "exact_ordered"`.

This path is fast and preserves the strongest evidence that a neighboring PDB directly describes the pickle array.

8.4 Unique spatial fallback

When ordered equality fails, the mapper groups atoms by normalized element and builds a coordinate nearest-neighbor index for each group. A pickle atom is accepted only when:

- its nearest same-element PDB atom lies within `structure.coordinate_match_tolerance_angstrom`;
- there is no equal-distance nearest-neighbor tie;
- no other pickle atom claims the same PDB atom.

An accepted fallback receives `spatial_unique`. Ties and collisions receive `ambiguous`; unmatched atoms receive `unresolved`.

This fallback is intentionally conservative. It does not use residue names or the four-character PDB ID to manufacture an atom correspondence.

8.5 Mapping quality versus geometry quality

These dimensions answer different questions:

Dimension	Question
Geometry quality	Is the source pickle internally valid and DrugCLIP-compatible?
Source mapping quality	Can its atoms be assigned to neighboring PDB atom metadata?
RCSB structure mapping quality	Can contributing local chains be mapped to validated RCSB/mmCIF entities?

A valid pickle can remain a usable candidate even when local structure mapping is incomplete. The uncertainty is surfaced in mapping columns and `processing_issues`; it does not rewrite the geometry.

9 Stable identities and hashes

9.1 Complex identity

A combine-set complex ID is derived from:

```
distribution ID + normalized directory PDB ID + data.pkl SHA-256
```

Thus replacing a pickle in place changes the complex identity even if the directory name stays the same.

9.2 Pocket content hash

The source-pickle content hash covers the exact ordered source tokens and little-endian float32 coordinates:

```
length-frame(
  "pocket-content-v2-source-pickle",
  NUL-joined ordered atom tokens,
  contiguous little-endian float32 coordinate bytes
)
```

The validator independently regenerates this hash from `pocket_atoms.parquet`.

9.3 Pocket derivation hash

The derivation hash adds the source checksum, source pocket name, explicit preservation policy, derivation schema, and content hash. The pocket instance ID is minted from the complex ID and derivation hash.

Content equality is not occurrence equality. Two occurrences with identical geometry can share a content hash while retaining distinct complex and pocket instance IDs.

10 Sidecar schema v2

10.1 Additive source-neutral fields

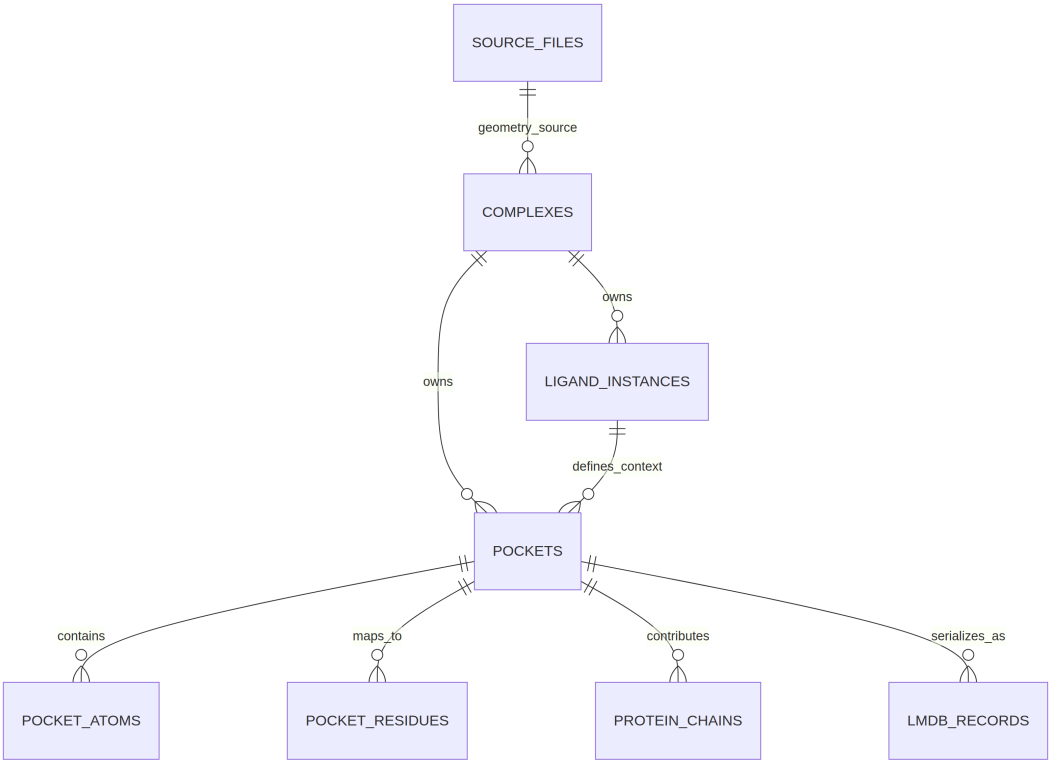
The original schema contains names tied to PDBbind re-extraction. Removing those columns would invalidate completed runs and downstream readers. Version 2 therefore evolves the schema additively.

Important additions include:

Table	Added fields	Purpose
`complexes`	`geometry_origin`, `geometry_source_file_id`	Identify the authoritative geometry source
`pockets`	`geometry_origin`, `geometry_source_file_id`, `derivation_method`, source atom counts	Explain how geometry entered the library
`pocket_atoms`	`source_atom_key`, `geometry_source_file_id`, `source_mapping_status`, `included_in_lmdb_source`	Separate source ordering and mapping from legacy PDBbind naming
`pocket_comparisons`	`left_geometry_role`, `right_geometry_role`	Name representations without relying on legacy count-column names
`lmdb_records`	`record_representation`	Select the correct serialization validation contract

New PDBbind and combine-set builds write schema v2. The reader and validator continue to accept v1 files when their only difference is the absence of these additive columns. Missing v2 fields are materialized as null during a compatibility read.

10.2 Core relationships



Primary keys, foreign keys, canonical sort order, and controlled enums are validated after writing.

11 Representation-aware LMDB export

11.1 Minimal record contract

Every candidate record contains exactly three fields:

```
{
    "pocket": pocket_instance_id,
    "pocket_atoms": ordered_atom_tokens,
    "pocket_coordinates": contiguous_little_endian_float32_array,
}
```

The original pickle bytes are not copied into the library. The record is reconstructed from validated sidecars and serialized with pickle protocol 4. This prevents unrelated ligand or label fields from leaking into the candidate LMDB and makes the output contract uniform.

11.2 Source-pickle export

For `drugclip_export_view = "source_pickle"`, the exporter:

1. selects atoms with `included_in_lmdb_source = true`;
2. orders them by `source_order`;
3. retains raw source tokens, including `H`;
4. converts coordinates deterministically to contiguous `<f4`;
5. permits an atom count greater than `pocket.max_pocket_atoms`;
6. verifies that DrugCLIP's token transformation reaches dictionary-supported tokens.

11.3 Post-transform export

For the existing PDBbind representation, the exporter:

1. selects `retained_after_crop = true`;
2. orders by `export_order`;
3. rejects hydrogen;
4. enforces the configured maximum atom count;
5. rejects lossy token transformation.

11.4 Dense keys and checksums

LMDB keys are dense ASCII integers:

```
0, 1, 2, ..., record_count - 1
```

Pocket instance IDs determine the canonical record order. Each `lmdb_records` row records:

- numeric key and record index;
- pocket instance and geometry hashes;
- representation;
- atom count;
- serialized record SHA-256;
- framed logical record SHA-256.

Profile metadata also records the LMDB physical checksum, logical checksum, source sidecar checksums, dictionary/task/loader/helper hashes, pickle protocol, and map size. Embedding caches must be namespaced or invalidated using the logical LMDB checksum.

12 Post-build RCSB and UniProt enrichment

12.1 Why enrichment is a separate workflow

The dedicated combine-set configuration is intentionally offline:

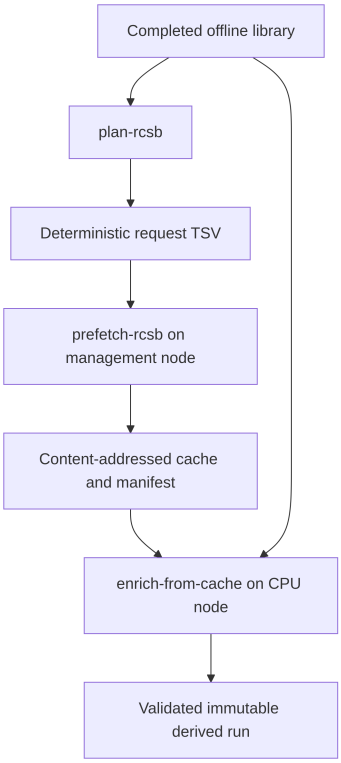
```
[pipeline]
offline = true

[rcsb]
download_mmcif = false
```

This is the appropriate default for Sagres because CPU and GPU compute nodes do not have internet access. More importantly, external-service availability should not determine whether the expensive geometry build succeeds. The implemented workflow therefore separates three concerns:

1. the completed offline library determines exactly which PDB IDs are needed;
2. a networked management node downloads and verifies those mmCIF files;
3. an offline CPU node derives a new enriched run from the verified cache.

The same commands accept either a combine-set run or a PDBbind run because both libraries publish the same normalized sidecar contract.



Only `prefetch-rcsb` needs internet access. `plan-rcsb` and `enrich-from-cache` contain no download step. None of the three commands needs or uses a GPU; parsing mmCIF and writing Parquet are CPU operations.

12.2 Request planning

`plan-rcsb` reads accepted complexes from a complete run and writes sorted, unique PDB IDs. The TSV begins with canonical JSON metadata containing the parent run ID, manifest checksum, stable identity fields, relevant logical sidecar hashes, and the requested count. The body is deliberately simple so it can be audited with ordinary text tools.

```
uv run biosensia-pocket-library plan-rcsb \
  --run-dir data/processed/drugclip_combine_set/<offline-run-id> \
  --output rcsb-request.tsv
```

The command refuses to replace an existing request unless `--overwrite` is given. Request planning requires a `complete` run, not a `sidecars_complete` run, because post-build enrichment preserves a published LMDB as part of the derived library.

12.3 Management-node prefetch

On a node with internet access, `prefetch-rcsb` downloads the exact request into the established content-addressed cache:

```
uv run biosensia-pocket-library prefetch-rcsb \
  --config config/drugclip-combine-set-library.toml \
  --request rcsb-request.tsv \
  --cache-dir /shared/biosensia/rcsb-cache
```

For this command, the CLI explicitly enables online operation even if the supplied build configuration is offline. Existing valid cache entries are reused. Each entry is accepted only when all of the following hold:

- its reference names the configured compressed or uncompressed form;
- the content-addressed object exists;
- the object's SHA-256 equals the reference checksum;
- decompression succeeds when required;
- Gemmi can parse the mmCIF;
- the mmCIF data-block identity equals the requested PDB ID.

After download, the command resolves every request again through the no-network cache validator. It writes a prefetch manifest below `<cache-dir>/manifests/rcsb_mmcif/` containing the request, portable relative cache paths, payload and reference checksums, retrieval metadata, failures, and a cache fingerprint. The command fails when any entry is missing by default; `--allow-partial` is an explicit exception for a deliberately partial cache.

12.4 Offline cache application

Once the cache is visible on a compute node, invoke:

```
uv run biosensia-pocket-library enrich-from-cache \
  --run-dir data/processed/drugclip_combine_set/<offline-run-id> \
  --cache-dir /shared/biosensia/rcsb-cache
```

The command automatically loads the parent run's `config.resolved.toml`, forces offline execution, and points external-cache resolution at `--cache-dir`. It validates the parent and every requested cache object before creating output. An incomplete cache is a hard error unless `--allow-partial` is supplied.

The original run is never edited. The derived run is written beside it by default, or below `--output-root`, with an ID of the form:

```
<parent-run-id>-rcsb-v1-<enrichment-identity12>
```

The enrichment identity covers the stable parent identity, relevant logical sidecar hashes, cache fingerprint, compression form, and partial-cache policy. The manifest records the parent manifest checksum and cache records, and states `network_access_used = false` for the enrichment step.

Unchanged Parquet and LMDB files are hard-linked when the filesystem permits and copied otherwise. Atomic replacement breaks links only for sidecars that must change. The LMDB payload is not regenerated: its physical and logical checksums remain identical to the parent. Its profile JSON is updated with the new `pockets.parquet` checksum and parent/cache provenance.

Cache-only enrichment can populate:

- chain mapping candidates and selected RCSB entity mappings;
- chain-to-UniProt mappings and residue ranges;
- ligand mapping candidates;
- structure citations and authors;
- structural-citation evidence for affinity-reference adjudication;
- RCSB cache objects and references in `source_files.parquet`.

The command rejects a parent that is already RCSB-enriched, preventing an ambiguous second application. With `--allow-partial`, absent or unparseable entries become `RCSB_ENRICHMENT_FAILED` issues and receive explicit `unavailable` status. In all cases, enrichment cannot change atom membership, coordinates, content hashes, LMDB record order, or LMDB eligibility.

12.5 The same workflow for PDBbind

No PDBbind-specific variant is needed. Substitute a completed PDBbind run in the planning and enrichment commands:

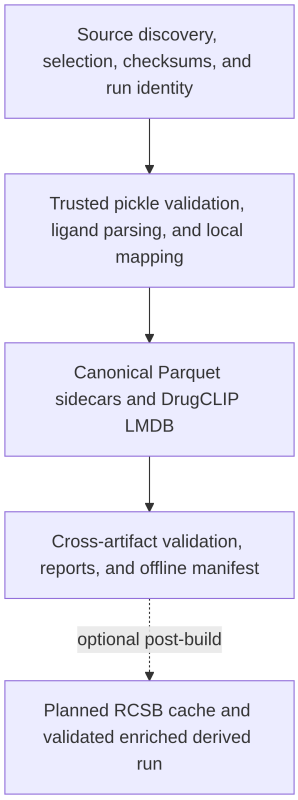
```
uv run biosensia-pocket-library plan-rcsb \
  --run-dir data/processed/pdbbind_2020_v2024p_20250804/<offline-run-id> \
  --output pdbbind-rcsb-request.tsv

uv run biosensia-pocket-library enrich-from-cache \
  --run-dir data/processed/pdbbind_2020_v2024p_20250804/<offline-run-id> \
  --cache-dir /shared/biosensia/rcsb-cache
```

The earlier inline online enrichment mode remains available for local workstations, but the staged workflow is the recommended operational path for both libraries on Sagres.

13 Build orchestration and run identity

13.1 Stage flow



13.2 Run directory name

The run ID exposes abbreviated fingerprints:

```
dc-combine-v1-<semantic8>-<source8>-<selection8>-<contract8>
```

The full values remain in `manifest.json`.

The semantic configuration hash covers content-affecting configuration, including trust/preservation policy and quality rules. Source paths do not enter that hash because selected contents are independently fingerprinted. Worker count and progress display are operational settings and do not alter logical outputs.

13.3 Resume and overwrite

`--resume` accepts an existing run only when semantic, source, selection, and DrugCLIP contract fingerprints all match. A complete compatible run is returned without rebuilding.

`--overwrite-run` does not delete an existing run. It first moves it to a timestamped sibling backup, then creates the new run at the identity-derived path.

14 Configuration guide

The supplied configuration is `config/drugclip-combine-set-library.toml`. Its essential source-specific settings are:

```toml
[paths]
combine_set_root = "BioSensIA-DC/external/DrugCLIP/data/pdb/combine_set"
combine_set_output_root = "data/processed/drugclip_combine_set"
drugclip_dir = "BioSensIA-DC/external/DrugCLIP"
drugclip_dictionary = "BioSensIA-DC/external/DrugCLIP/data/dict_pkt.txt"

[combine_set]
trusted_pickles = true
distribution_id = "drugclip-combine-set"
nominal_release = "unversioned-vendored-snapshot"
preserve_source_geometry = true
compare_coordinate_tolerance_angstrom = 0.0001
```

`preserve_source_geometry = false` is rejected. The current builder has no mode that silently filters or crops source pickle geometry.

15 Operating guide

15.1 1. Install and verify the environment

```
uv sync

uv run biosensia-pocket-library check-drugclip-contract \
  --config config/drugclip-combine-set-library.toml
```

The contract check fingerprints the pocket dictionary, DrugCLIP task, relevant loader wrappers, and BioSensIA LMDB helper. It reports but does not hash the encoder checkpoint because the checkpoint is not a library-construction input.

15.2 2. Inventory without deserializing

```
uv run biosensia-pocket-library inventory-combine-set \
  --config config/drugclip-combine-set-library.toml \
  --pdb-id 2h2h
```

The implementation-time `2h2h` inventory reported one selected bundle, six files, and no inventory issues.

15.3 3. Build a small audit selection

```
uv run biosensia-pocket-library build-combine-set \
  --config config/drugclip-combine-set-library.toml \
  --limit 10
```

Use a small deterministic selection before a full run to verify local paths, available memory, ligand parsing behavior, and expected mapping quality.

15.4 4. Build sidecars without LMDB

```
uv run biosensia-pocket-library build-combine-set-sidecars \
  --config config/drugclip-combine-set-library.toml \
  --pdb-ids-file selected-pdb-ids.txt
```

This produces a `sidecars_complete` run. The shared `finalize` command can later export the default LMDB and finish validation and reporting.

15.5 5. Build the complete selected library

```
uv run biosensia-pocket-library build-combine-set \
  --config config/drugclip-combine-set-library.toml \
  --pdb-ids-file selected-pdb-ids.txt
```

Omit all selection arguments to select every discovered `data.pkl` occurrence.

15.6 6. Validate independently

```
uv run biosensia-pocket-library validate \
  --run-dir data/processed/drugclip_combine_set/<run-id>
```

15.7 7. Plan the optional external request

This step can run on the same offline CPU node that built the library:

```
uv run biosensia-pocket-library plan-rcsb \
  --run-dir data/processed/drugclip_combine_set/<run-id> \
  --output rcsb-request.tsv
```

15.8 8. Prefetch on the management node

Move only the request file to the internet-connected management node if the library filesystem is not mounted there. Then populate a shared or transferable cache:

```
uv run biosensia-pocket-library prefetch-rcsb \
  --config config/drugclip-combine-set-library.toml \
  --request rcsb-request.tsv \
  --cache-dir /shared/biosensia/rcsb-cache
```

If `/shared` is unavailable to compute nodes, transfer the entire cache root, including `objects/`, `request_index/`, and `manifests/`, to node-local or job storage. Relative paths in the prefetch manifest make the cache relocatable.

15.9 9. Derive the enriched library offline

Run this on a CPU node. A GPU allocation provides no benefit.

```
uv run biosensia-pocket-library enrich-from-cache \
  --run-dir data/processed/drugclip_combine_set/<run-id> \
  --cache-dir /shared/biosensia/rcsb-cache
```

The command prints the derived run path. Preserve the printed path rather than assuming the identity suffix.

15.10 10. Validate the derived run independently

```
uv run biosensia-pocket-library validate \
  --run-dir data/processed/drugclip_combine_set/<derived-run-id>
```

15.11 11. Regenerate reports

```
uv run biosensia-pocket-library report \
  --run-dir data/processed/drugclip_combine_set/<derived-run-id>
```

16 Output layout

```
data/processed/drugclip_combine_set/<run-id>/
   config.resolved.toml
   manifest.json
   checkpoints/
   logs/
   sidecars/
       source_files.parquet
       complexes.parquet
       ligand_instances.parquet
       pockets.parquet
       pocket_atoms.parquet
       pocket_residues.parquet
       protein_chains.parquet
       pocket_comparisons.parquet
       processing_issues.parquet
       lmdb_records.parquet
       ...
   lmdb/
       candidate_pockets.lmdb
       candidate_pockets.lmdb.profile.json
   reports/
        build_summary.json
        build_summary.md
        *.parquet
```

Coordinate-bearing outputs inherit the source data's access and redistribution constraints and must not be committed or redistributed without authorization.

17 Validation model

Validation is layered so that a checksum alone cannot make an invalid library appear sound.

17.1 Source-record validation

- explicitly authorized pickle execution;
- dictionary record type;
- required pocket fields;
- one finite pocket coordinate set;
- equal atom and coordinate counts;
- nonempty pocket name matching the directory;
- transformed token coverage by the active dictionary.

17.2 Sidecar validation

- exact current schema or compatible additive v1 schema;
- canonical row order;
- primary-key uniqueness;
- controlled enum values;
- foreign-key integrity;
- accepted pockets have atom rows;

- derivation hashes map consistently to content hashes;
- regenerated geometry hashes match pocket rows.

17.3 LMDB validation

- dense numeric keys;
- entry count equals `lmdb_records` count;
- exact three-field record shape;
- pocket name equals the sidecar-ordered pocket instance ID;
- contiguous little-endian float32 coordinates;
- finite `(n_atoms, 3)` geometry;
- representation-specific hydrogen, token, and atom-count rules.

17.4 Manifest and artifact validation

- run-directory name equals manifest run ID;
- source, selection, semantic, and DrugCLIP contract fingerprints are recorded;
- profile metadata contains sidecar and LMDB hashes;
- final artifact inventory records physical file sizes and SHA-256 values.

17.5 Post-build enrichment validation

- request IDs are canonical, unique, sorted, and count-consistent;
- cache references and objects are revalidated without network access;
- the parent run must be complete, valid, and not already RCSB-enriched;
- the derived manifest records parent, cache, code, and configuration provenance;
- changed sidecars are rewritten atomically and all foreign keys are rechecked;
- the derived LMDB checksum must remain the reused parent checksum;
- the complete derived run passes the same relational and LMDB validator as a directly built library.

18 Test strategy and observed evidence

18.1 Synthetic tests

The synthetic end-to-end fixture deliberately creates a source pickle with:

- atoms `C`, `H`, and `N`;
- three coordinates;
- a configured post-transform maximum of two atoms;
- an exactly matching pocket PDB;
- a valid ligand SDF;
- a DrugCLIP affinity label.

The test proves that:

- trust is mandatory;
- nonfinite pickle coordinates are rejected;
- exact ordered mapping succeeds;
- the source atom count remains three;
- hydrogen remains present in the serialized LMDB;

- the two-atom post-transform limit does not reject a `source_pickle` record;
- the serialized coordinate array is contiguous little-endian float32;
- complete run validation succeeds.

Additional tests cover CLI parsing, automatic LMDB map growth, schema-v1 compatibility, existing PDBbind behavior, deterministic worker-count outputs, finalization, source-neutral PDBbind request planning, deterministic request round trips, cache-manifest generation, offline enrichment, parent immutability, LMDB checksum reuse, strict incomplete-cache failure, and checksum-tampering rejection.

At the time this document was written, the complete test suite reported:

```
37 passed
```

18.2 Real-source verification

The final implementation was exercised against the real `2h2h` bundle. The run reported:

Metric	Value
Source pickle atoms	480
Source heavy atoms	238
Hydrogen retained in candidate LMDB	yes
Atoms mapped to neighboring PDB	480
Local mapping quality	exact
Pocket comparison quality	concordant
LMDB record representation	`source_pickle`
Independent run validation	passed

These values are verification evidence for that source occurrence, not global assumptions about all 18,938 discovered pickles.

19 Performance and scaling

19.1 Bounded processing concurrency

`pipeline.workers` controls source-record processing. The bounded scheduler submits at most that many jobs and yields completed jobs without waiting for an earlier slow record. Canonical sidecar sorting ensures that worker completion order does not change logical outputs.

19.2 Memory planning

The current shared sidecar architecture materializes selected normalized rows before the atomic Parquet write and reads atom sidecars during LMDB export and validation. Raw combine-set pockets can contain many atoms, including hydrogen. Consequently, a full-source run needs materially more memory and storage than a small audit selection.

Before a full run:

1. build representative `--limit` selections;
2. inspect pocket-size reports and peak memory;
3. confirm output and temporary filesystem capacity;

4. choose workers based on memory, not CPU count alone;
5. retain the run's resolved configuration and inventory output.

This is an operational scaling constraint, not a reason to discard source atoms. A future streaming sidecar writer must preserve the same ordering, hashing, relational, and atomic-publication contracts.

19.3 Compute and network placement

The geometry build and post-build enrichment are CPU workloads. They use threaded parsing and hashing but contain no CUDA path. Request planning is lightweight. Prefetch is network-bound with modest CPU use for decompression and mmCIF validation. On Sagres, use this placement:

Step	Recommended node	Internet	GPU
combine-set or PDBbind build	CPU compute	no	no
`plan-rcsb`	CPU compute or management	no	no
`prefetch-rcsb`	management	yes	no
`enrich-from-cache`	CPU compute	no	no

The cache can be populated before submitting the enrichment job. The job then depends only on local/shared files and cannot accidentally contact RCSB.

20 Troubleshooting

20.1 Refusal to deserialize pickle

```
combine_set builds require combine_set.trusted_pickles=true
```

Confirm that the configured root is the trusted vendored DrugCLIP snapshot, then use the dedicated configuration or the explicit CLI flag. Do not bypass the check for an unknown source.

20.2 Pickle shape or finiteness error

The record is rejected and listed in `processing_issues.parquet`. Do not reshape or truncate the record manually. Determine whether the source snapshot is corrupt or whether a new coordinate layout needs an explicit, tested parser.

20.3 Dictionary lacks transformed tokens

The active `dict_pkt.txt` is incompatible with the selected source record or the configured DrugCLIP checkout. Verify the symlink, contract fingerprint, and dictionary provenance before considering a token policy change.

20.4 Incomplete atom mapping

Inspect:

- `pocket_atoms.source_mapping_status`;
- `pocket_comparisons.left_geometry_role` and `right_geometry_role`;
- coordinate tolerances in resolved configuration;
- the neighboring `pocket.pdb` and `pocket6A.pdb` source checksums.

An incomplete mapping does not imply invalid pickle geometry. It means that residue or chain annotations are incomplete and must not be guessed.

20.5 Existing run already present

Use `--resume` only for an identical run identity. Use `--overwrite-run` to preserve the old directory as a timestamped backup before rebuilding.

20.6 RCSB cache is incomplete

Re-run `prefetch-rcsb` on the management node using the same request and cache root. The command reuses valid objects and downloads only unresolved entries. Inspect the written prefetch manifest for per-PDB failure details. Use `--allow-partial` only when unavailable mappings are an intentional, recorded outcome.

20.7 Parent run is already RCSB-enriched

Post-build enrichment is a one-step derivation from an offline parent. Select the original offline run instead of chaining enrichment onto an already enriched run. This keeps parent/cache provenance and failure semantics unambiguous.

20.8 Cache checksum or mmCIF identity failure

Treat the object as corrupt or incorrectly indexed. Do not edit its reference JSON to bypass validation. Remove or quarantine the bad cache entry on the management node and run `prefetch-rcsb --refresh-cache` to retrieve and verify a replacement.

20.9 Validation reports a content-hash mismatch

The atom sidecar and pocket row disagree. Treat the run as invalid. Do not edit Parquet or LMDB artifacts in place; rebuild from the fingerprinted sources.

21 Scientific interpretation

This library is a provenance-preserving projection of DrugCLIP combine-set pocket inputs. It does not claim that every neighboring structure file is an independent experimental validation of the pickle geometry, nor that a PDB ID uniquely identifies a binding site or chain.

The implementation deliberately keeps these claims separate:

- source pickle geometry is preserved as model input;
- local PDB mapping supplies evidence-backed structural annotations;
- SDF/MOL2 parsing supplies ligand identity;
- RCSB supplies optional external structure and UniProt mappings;
- content hashes identify geometry bytes;
- instance IDs identify retained source occurrences;
- quality and mapping statuses expose uncertainty without silently changing the geometry.

That separation is the central design principle of the second candidate-pocket library.